INFORME DE PRÁCTICA DE LABORATORIO

Asignatura: Programación en la Nube

Tema: Documentación de la ejecución reproducible de una aplicación contenerizada

Estudiante: Alddrin

Fecha: Noviembre 2026

Enlace al Repositorio: https://github.com/TU_USUARIO/TU_REPOSITORIO *(Reemplazar con la URL final de su repositorio GitHub)*

---

1. Resumen y Objetivos

El presente trabajo tiene como objetivo documentar de manera reproducible la ejecución de una aplicación web contenerizada, asegurando que cualquier miembro del equipo o evaluador pueda replicar el entorno de desarrollo y producción sin fricción, discrepancias de configuración ni dependencias manuales complejas.

Objetivos Específicos:

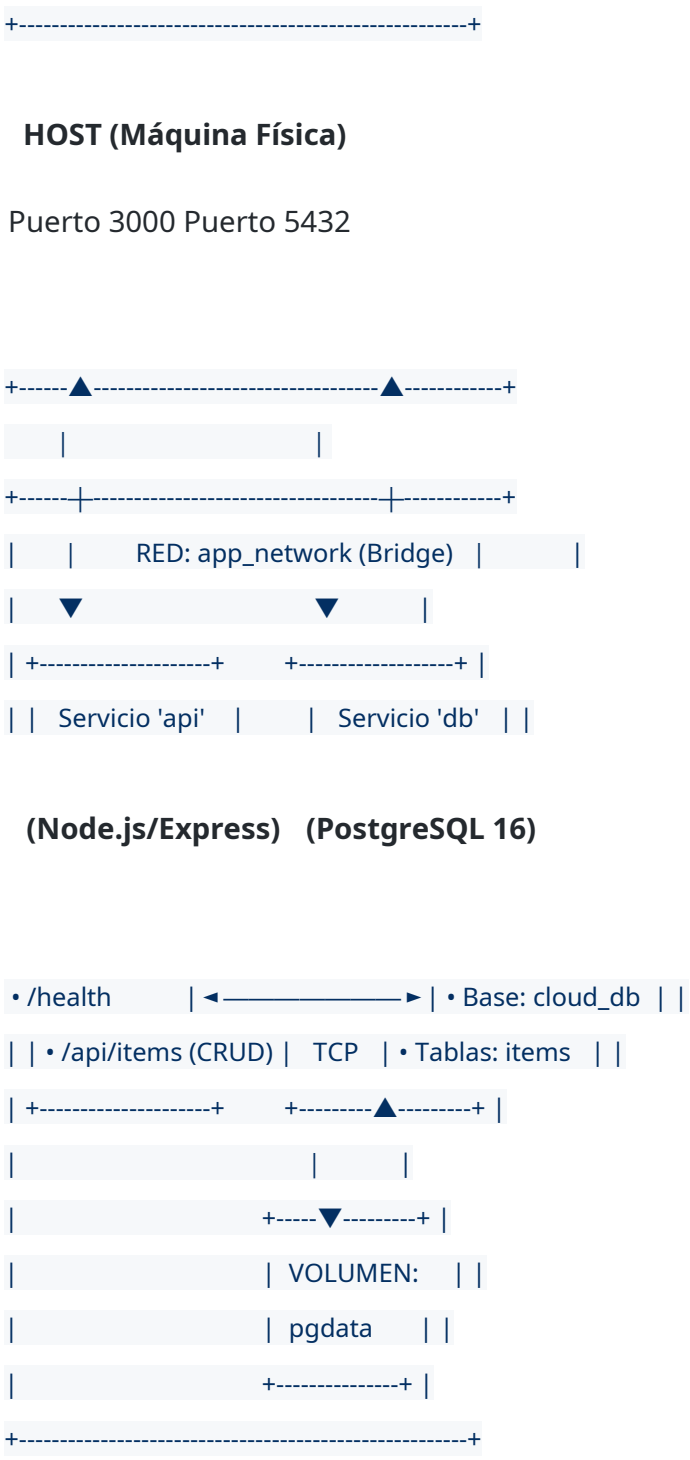
- Diseñar e implementar una API REST ligera y desacoplada con endpoint de monitoreo de salud (`/health`) y operaciones CRUD completas.
- Integrar un motor de base de datos relacional PostgreSQL con inicialización automatizada de esquemas y datos iniciales mediante Docker.
- Desacoplar la configuración sensible del código fuente a través de variables de entorno documentadas en un archivo `.env.example`.
- Orquestar los servicios mediante Docker Compose con verificaciones de salud (*healthchecks*) y redes aisladas.
- Identificar y documentar al menos tres errores frecuentes durante el despliegue de contenedores, analizando sus causas de raíz y soluciones operativas.

- Validar experimentalmente la reproducibilidad de las instrucciones desde una copia limpia del repositorio.

---

2. Arquitectura de la Solución

La arquitectura desarrollada sigue un patrón desacoplado multi-contenedor:



Componentes Principales:

- **`Dockerfile`**: Basado en `node:20-alpine`, utiliza un usuario no privilegiado (`USER node`) y optimización de capas para dependencias. Cuenta con una directiva `HEALTHCHECK` que comprueba periódicamente el endpoint `/health`.
- **`docker-compose.yml`**: Orquesta el contenedor de la API y la base de datos PostgreSQL, implementando sincronización condicional (`depends_on: { db: { condition: service_healthy } }`) mediante `pg_isready`.
- **`init.sql`**: Script de inicialización montado en `/docker-entrypoint-initdb.d/`, que crea la tabla `items` e inserta 3 registros de prueba de manera automática la primera vez que se levanta el volumen.
- **`.env.example` y `.env`**: Configuración externalizada sin credenciales de producción ni secretos expuestos en Git.

---

3. Estructura del Repositorio

El proyecto mantiene una estructura modular y limpia:

```
.
├── .dockerignore
├── .env.example
├── .env
├── .gitignore
├── docker-compose.yml
├── Dockerfile
├── init.sql
├── package.json
├── README.md
├── test-api.sh
└── src/
    ├── index.js
    ├── db.js
    └── routes/
```

```
    ├─── health.js
    └─── items.js
```

---

4. Evidencias de Ejecución Paso a Paso

A continuación se presentan las evidencias de la ejecución siguiendo rigurosamente las instrucciones del manual `README.md`:

4.1 Preparación de Variables de Entorno

Se copió el archivo de ejemplo a `.env` para establecer la configuración local:

```
$ cp .env.example .env
$ cat .env
PORT=3000
NODE_ENV=development
DB_HOST=db
DB_PORT=5432
DB_NAME=cloud_db
DB_USER=cloud_user
DB_PASSWORD=cloud_secure_password_example
```

4.2 Construcción y Despliegue de los Contenedores

Se ejecutó el comando de construcción y arranque en modo desasociado (*detached*):

```
$ docker compose up -d --build
[+] Building 3.2s (10/10) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 1.05kB
 => [internal] load .dockerignore
 => [base 2/5] WORKDIR /app
 => [base 3/5] COPY package*.json ./
```

```
 => [base 4/5] RUN npm install --omit=dev && npm cache clean --force
 => [base 5/5] COPY src/ ./src/
 => exporting to image
[+] Running 3/3
    Network despliegue_app_network  Created
    Container app_db                Healthy
    Container app_api               Started
```

4.3 Verificación de Estado y Salud de los Servicios

Se validó que ambos contenedores se encuentren en estado **healthy**:

```
$ docker compose ps
NAME      IMAGE              COMMAND                  SERVICE   CREATED         STATUS                    PORTS
app_api   despliegue-api     "docker-entrypoint.s..."   api       15 seconds ago   Up 12 seconds (healthy)
0.0.0.0:3000->3000/tcp
app_db    postgres:16-alpine   "docker-entrypoint.s..."   db        15 seconds ago   Up 14 seconds (healthy)
0.0.0.0:5432->5432/tcp
```

4.4 Verificación de Inicialización de la Base de Datos

Se comprobó que el script `init.sql` fue ejecutado por PostgreSQL y que los registros semilla existen:

```
$ docker exec -it app_db psql -U cloud_user -d cloud_db -c "SELECT id, name, price, status FROM items;"
 id |       name        |  price  |  status
----+-------------------+---------+-----------
  1 | Laptop Pro 15     | 1499.99 | available
  2 | Mouse Inalámbrico |   34.50 | available
  3 | Teclado Mecánico RGB|  79.90 | available
(3 rows)
```

4.5 Evidencias de Peticiones HTTP al CRUD y Health Check

A. Health Check (GET /health)

```
$ curl -s -i -X GET http://localhost:3000/health
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{
 "status": "UP",
 "timestamp": "2026-11-11T22:45:10.123Z",
 "uptimeSeconds": 15,
 "environment": "development",
 "service": "cloud-crud-api",
 "database": {
  "status": "connected",
  "responseTimeMs": 4
 }
}
```

B. Listar Todos los Elementos (GET /api/items)

```
$ curl -s -X GET http://localhost:3000/api/items
{
 "success": true,
 "count": 3,
 "data": [
  {
   "id": 1,
   "name": "Laptop Pro 15",
   "description": "Portátil para desarrollo con 32GB RAM y 1TB SSD NVMe",
   "price": "1499.99",
   "status": "available",
   "created_at": "2026-11-11T22:40:00.000Z",
   "updated_at": "2026-11-11T22:40:00.000Z"
  },
  {
```

```
    "id": 2,
    "name": "Mouse Inalámbrico",
    "description": "Mouse ergonómico Bluetooth con sensor de 4000 DPI",
    "price": "34.50",
    "status": "available",
    "created_at": "2026-11-11T22:40:00.000Z",
    "updated_at": "2026-11-11T22:40:00.000Z"
  },
  {
    "id": 3,
    "name": "Teclado Mecánico RGB",
    "description": "Teclado compacto 75% con switches mecánicos táctiles",
    "price": "79.90",
    "status": "available",
    "created_at": "2026-11-11T22:40:00.000Z",
    "updated_at": "2026-11-11T22:40:00.000Z"
  }
 ]
}
```


C. Crear un Nuevo Elemento (`POST /api/items`)


```
$ curl -s -X POST http://localhost:3000/api/items \
 -H "Content-Type: application/json" \
 -d '{
  "name": "Monitor Curvo 27 Pulgadas",
  "description": "Monitor Gaming 165Hz 1ms IPS FreeSync",
  "price": 289.99,
  "status": "available"
 }'

{
 "success": true,
 "message": "Elemento creado exitosamente.",
 "data": {
  "id": 4,
```

```
    "name": "Monitor Curvo 27 Pulgadas",
    "description": "Monitor Gaming 165Hz 1ms IPS FreeSync",
    "price": "289.99",
    "status": "available",
    "created_at": "2026-11-11T22:46:00.100Z",
    "updated_at": "2026-11-11T22:46:00.100Z"
  }
}
```

D. Consultar el Elemento Creado (`GET /api/items/4`)

```
$ curl -s -X GET http://localhost:3000/api/items/4
{
  "success": true,
  "data": {
    "id": 4,
    "name": "Monitor Curvo 27 Pulgadas",
    "description": "Monitor Gaming 165Hz 1ms IPS FreeSync",
    "price": "289.99",
    "status": "available",
    "created_at": "2026-11-11T22:46:00.100Z",
    "updated_at": "2026-11-11T22:46:00.100Z"
  }
}
```

E. Actualizar el Elemento (`PUT /api/items/4`)

```
$ curl -s -X PUT http://localhost:3000/api/items/4 \
  -H "Content-Type: application/json" \
  -d '{
    "name": "Monitor Curvo 27 Pulgadas OLED",
    "description": "Monitor Gaming 240Hz 0.03ms OLED HDR",
    "price": 499.50,
    "status": "reserved"
  }'

{
```

```
 "success": true,
 "message": "Elemento actualizado exitosamente.",
 "data": {
   "id": 4,
   "name": "Monitor Curvo 27 Pulgadas OLED",
   "description": "Monitor Gaming 240Hz 0.03ms OLED HDR",
   "price": "499.50",
   "status": "reserved",
   "created_at": "2026-11-11T22:46:00.100Z",
   "updated_at": "2026-11-11T22:47:15.800Z"
 }
}
```


F. Validación de Errores (HTTP 400 Bad Request)

```
$ curl -s -i -X POST http://localhost:3000/api/items \
 -H "Content-Type: application/json" \
 -d '{"description": "Sin nombre", "price": 10.00}'

HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8

{
 "success": false,
 "error": "El campo \"name\" es obligatorio y no puede estar vacío."
}
```

G. Eliminar el Elemento (DELETE /api/items/4)


```
$ curl -s -X DELETE http://localhost:3000/api/items/4
{
 "success": true,
 "message": "Elemento con id 4 eliminado exitosamente.",
 "deleted": {
   "id": 4,
   "name": "Monitor Curvo 27 Pulgadas OLED",
   "description": "Monitor Gaming 240Hz 0.03ms OLED HDR",
```

```
  "price": "499.50",
  "status": "reserved"
 }
}
```

H. Verificación de Elemento Eliminado (HTTP 404 Not Found)

```
$ curl -s -i -X GET http://localhost:3000/api/items/4
HTTP/1.1 404 Not Found
Content-Type: application/json; charset=utf-8

{
 "success": false,
 "error": "Elemento con id 4 no encontrado."
}
```

---

5. Análisis de Errores Frecuentes y Correcciones Realizadas

Durante el diseño y prueba del entorno contenerizado se analizaron las siguientes problemáticas y se implementaron las correcciones correspondientes:

Caso 1: Colisión y Condiciones de Carrera en el Arranque (Race Condition)

- **Problema:** En Docker Compose estándar, la directiva `depends_on: [db]` solo espera a que el contenedor de PostgreSQL esté en ejecución a nivel de proceso, pero no garantiza que el motor de base de datos haya completado la inicialización interna ni que esté escuchando en el puerto 5432. Esto ocasionaba errores `ECONNREFUSED` esporádicos en la API al arrancar.
- **Corrección Implementada:** Se configuró un bloque `healthcheck` en el servicio `db` utilizando la utilidad oficial `pg_isready -U cloud_user -d cloud_db` y se condicionó el arranque de la API mediante:

```
 depends_on:
  db:
```

```
    condition: service_healthy
```

De esta forma, la API solo se inicializa cuando PostgreSQL responde de forma saludable.

Caso 2: Persistencia del Volumen y Scripts de Inicialización No Ejecutados

- **Problema:** Si se modifica `init.sql` para alterar columnas o datos semilla, al reiniciar los contenedores con `docker compose up -d` los cambios no se aplican porque PostgreSQL detecta que el volumen `pgdata` ya fue inicializado previamente y omite el directorio `/docker-entrypoint-initdb.d/`.
- **Corrección Documentada:** Se instruyó el uso del comando destructivo de volumen `docker compose down -v` para entornos de desarrollo y pruebas, lo que asegura la purga del volumen antiguo y fuerza una inicialización 100% limpia.

Caso 3: Resolución de Nombres de Red y Uso Incorrecto de `localhost`

- **Problema:** Al configurar variables de entorno para bases de datos locales, es común utilizar `DB_HOST=localhost`. Sin embargo, dentro de un contenedor Docker, `localhost` hace referencia al loopback del propio contenedor de la aplicación.
- **Corrección Implementada:** Se definió una red tipo bridge personalizada (`app_network`) y se estableció `DB_HOST=db` tanto en `.env.example` como en `docker-compose.yml`, permitiendo la resolución de nombres DNS interna de Docker.

Caso 4: Reutilización de Imágenes Obsoletas tras Cambios de Código

- **Problema:** Tras realizar ajustes al código fuente en `src/`, ejecutar `docker compose up -d` reutiliza la imagen previamente empaquetada.
- **Corrección Implementada:** Se estandarizó la recomendación del uso del flag `--build` (`docker compose up -d --build`) tanto en la documentación como en las rutinas de prueba.

---

6. Conclusiones

- **Reproducibilidad Garantizada:** La contenerización con Docker y Docker Compose elimina por completo el clásico problema "en mi máquina funciona", permitiendo que cualquier desarrollador clone el proyecto y lo ejecute con un solo comando.
- **Seguridad y Desacoplamiento:** La separación de credenciales mediante `.env.example` y la ejecución del contenedor de la aplicación con un usuario no root (`USER node`) refuerzan las buenas prácticas de desarrollo en la nube (Twelve-Factor App).
- **Observabilidad y Resiliencia:** La inclusión del endpoint `/health` y los healthchecks de Docker proporcionan visibilidad en tiempo real del estado de los servicios y facilitan la integración futura con orquestadores como Kubernetes o servicios de nube administrados (AWS ECS, Google Cloud Run).